



LectureAssist: An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

Iftikhar Ahmed

iftikhar.ahmed@northsouth.edu

Salman Shafi

salman.shafi@northsouth.edu

Jarinah Tasnim

jarinah.tasnim@northsouth.edu

Dept. of Electrical and Computer Engineering, North South University • Senior Design Project (499)

Introduction

Recorded lectures pack speech, slides and board work into a single video, but that content stays **locked and unsearchable**. We present **LectureAssist**, a multimodal agentic system that extracts, indexes and reasons over a lecture to produce grounded, adaptive study material at **zero API cost** on a free Colab T4 GPU.

Motivation & Problem

Students re-watch hours of recorded lectures to find a single concept, and have no fast way to self-test on what the teacher actually emphasised.

- Lecture knowledge is locked inside **video, speech and slides**, rarely available as clean text.
- Generic quiz tools **hallucinate** off-topic, ungrounded questions.
- One-size-fits-all difficulty ignores **each student's weak spots**.

Goal. An offline-capable system that *reads the board, hears the lecture*, and produces *grounded, validated and adaptive* study tools at **zero API cost**.

Key Contributions & Why It's Better

- **Validation-gated, one-at-a-time quiz generation** that eliminates hallucinated, off-topic and duplicate questions, *better than* generic LLM quiz tools that emit a whole batch with no per-question check.
- **Fully local, \$0-API** multi-agent pipeline on a single free T4 GPU (Ollama + Gemma 3), *better than* cloud-API tools that incur per-call cost and require internet access.
- **Multimodal extraction** with per-surface OCR preprocessing for slides, blackboards and whiteboards, *better than* plain-transcript tools that miss board/slide content entirely.
- **Adaptive learning** grounded with clickable lecture **timestamps**, *better than* static quiz banks that ignore each student's weak topics and cite no source.

End-to-End Pipeline

A lecture video or document flows left-to-right through extraction, a grounded knowledge base, the agent orchestrator, and finally validated study outputs.

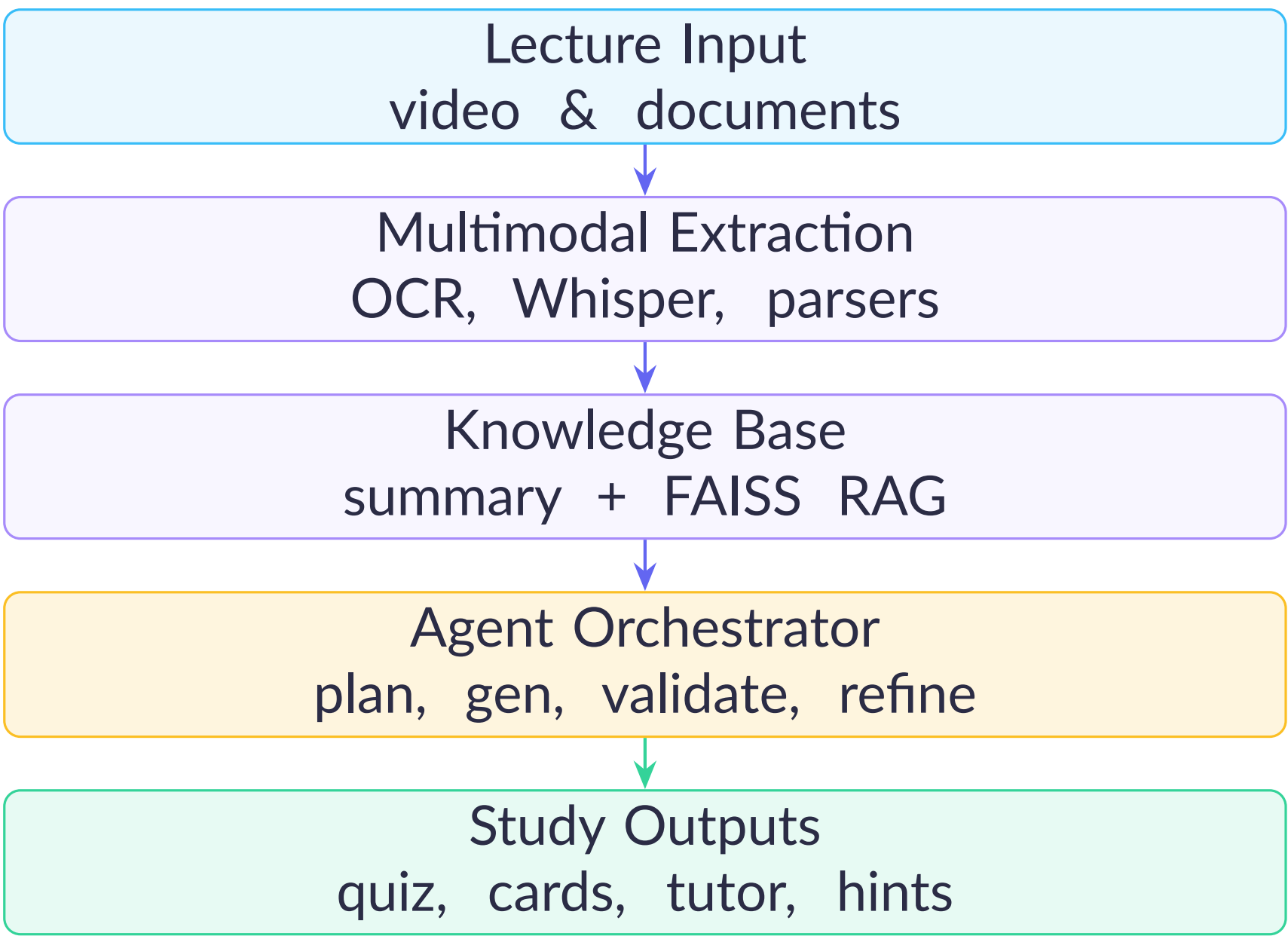


Figure 1. System pipeline: Flask on Colab → ngrok → browser UI.

Methodology

1. Multimodal Extraction

Every modality is recovered and time-aligned *before* any LLM call.

- **Visual / OCR (EasyOCR)**. Frames sampled ~ 5 s apart, *classified* (slide / blackboard / whiteboard / scene) with per-class preprocessing (CLAHE, sharpening, Otsu, chalk inversion). **EN**.
- **Speech (Faster-Whisper)**. GPU `float16` / CPU `int8` fallback, per-segment timestamps.
- **Documents**. PyMuPDF, python-docx, python-pptx (structure kept).
- **Drive OCR cache** keyed by video hash, re-uploads skip re-scan.

2. Multi-Agent Orchestration

A central **Orchestrator** runs specialist agents; independent agents run **concurrently**. Each follows `should_run` → `plan` → `execute` → `validate` → `retry`, and any crash is caught and logged so the pipeline **degrades gracefully**.

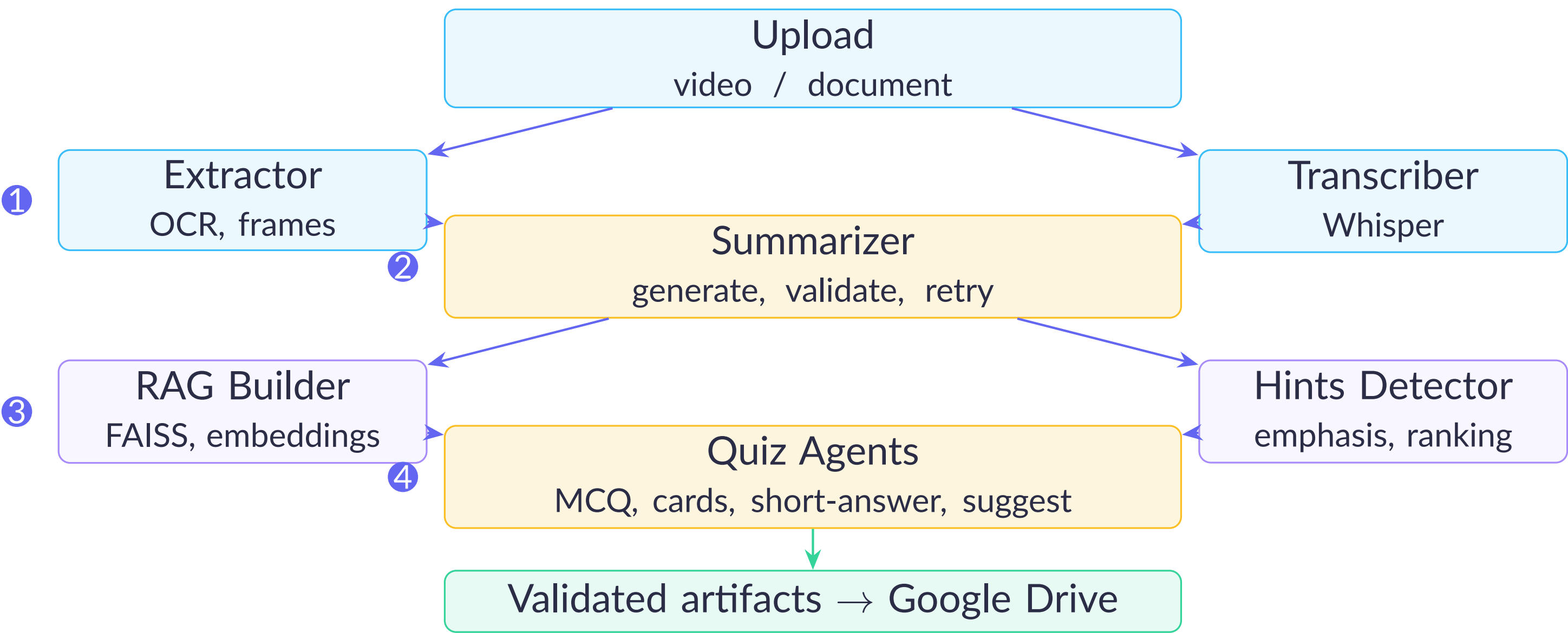


Figure 2. Pipeline stages **1–4**: Extractor/Transcriber → Summarizer → RAG Builder/Hints Detector → Quiz Agents. The Orchestrator supervises and validates every stage; any agent crash is isolated.

3. Grounded Retrieval (RAG)

Speech and board/slide text are embedded with **Sentence-Transformers (MiniLM)** and indexed in **FAISS** (cosine). Every quiz, summary and chat answer retrieves **top- k** chunks first, anchored to real content with a clickable timestamp.

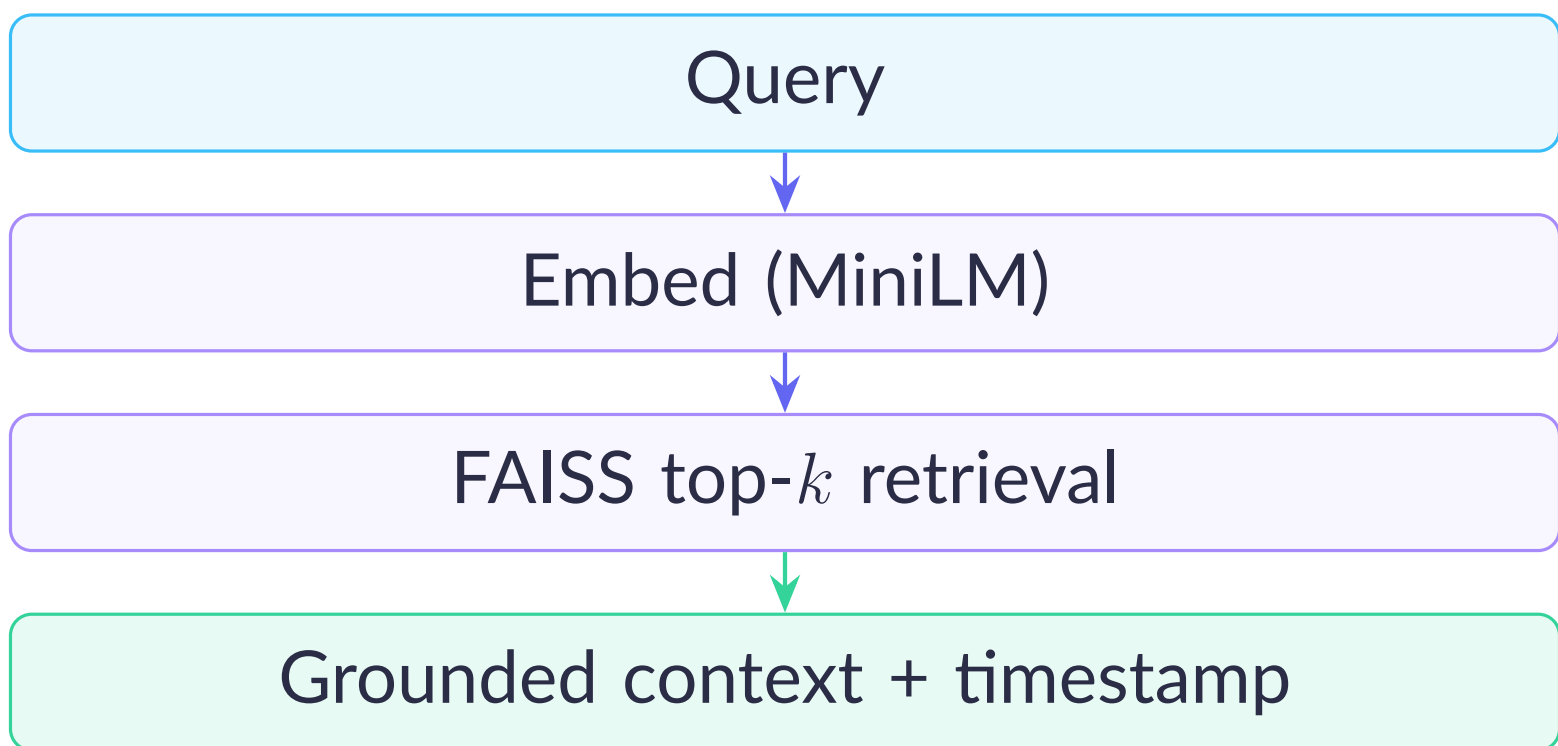


Figure 3. Retrieval path: every generated answer is grounded before it reaches the student.

Agentic Quiz Generation

Questions are built **one at a time** and must *pass validation* before the next is generated, eliminating hallucinated, off-topic or duplicate questions. Modes: *Auto*, *Manual* (Easy / Med / Hard), *Adaptive* (targets weak topics).

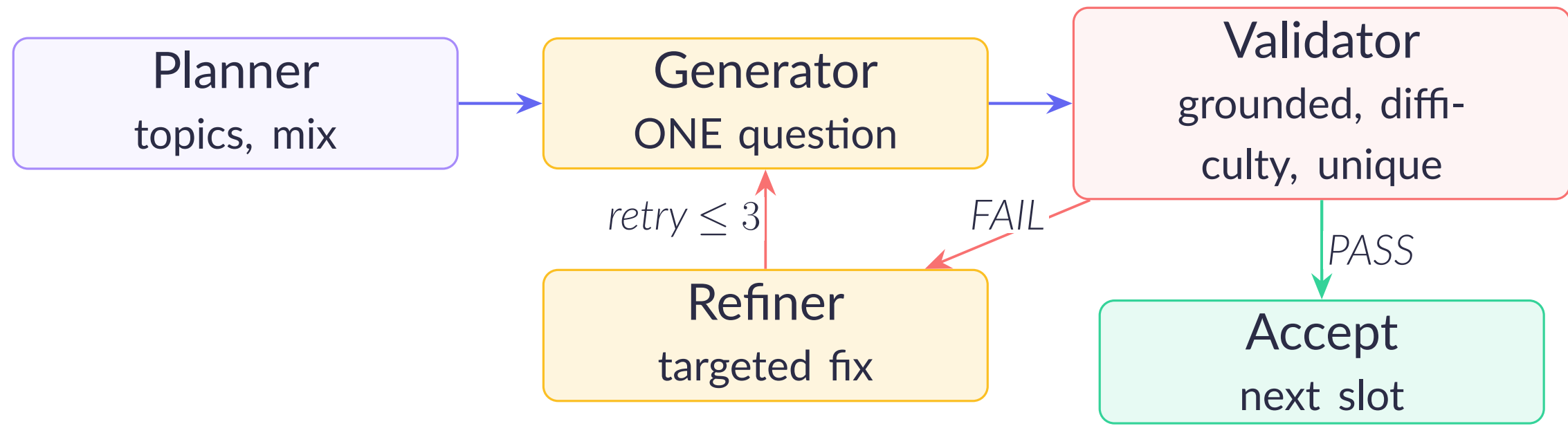


Figure 4. Generate → validate → refine loop (≤ 3 retries per slot).

Validation gate: grounded in lecture, correct difficulty, follows instructions, not a duplicate ($>85\%$ similarity rejected).

Adaptive Learning

- **Self-improving difficulty**. A Difficulty Adapter tracks per-topic accuracy, flags weak topics ($<50\%$) and steers the planner there.
- **AI short-answer grading** with partial credit and feedback.

User Interface

A lightweight **browser UI** talks to the Colab backend over **Flask + ngrok**: upload a lecture, then browse the summary, quizzes and grounded chat tutor, each citing a clickable **timestamp**.

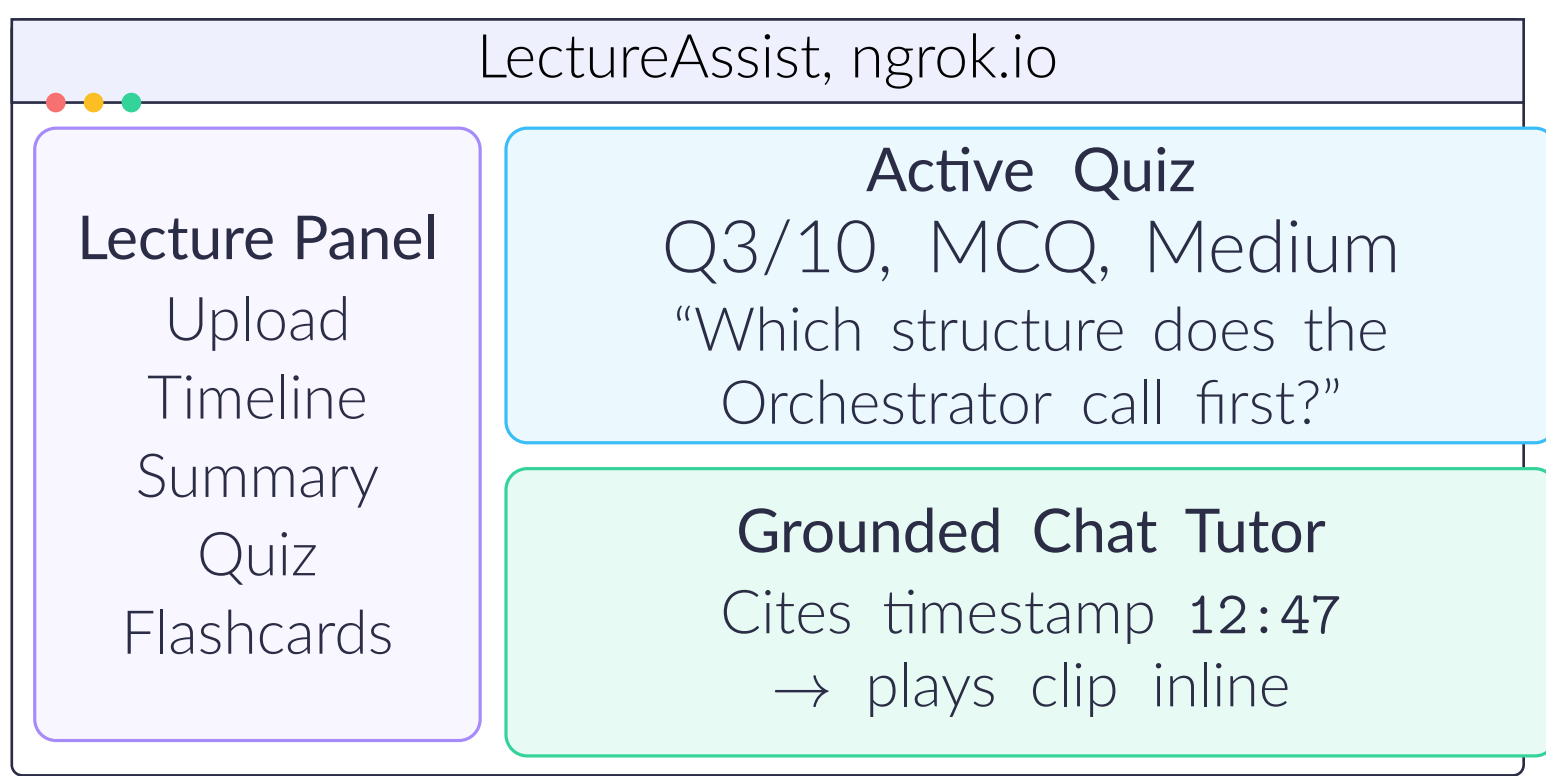


Figure 5. UI mock-up: lecture panel, live quiz, and grounded chat tutor.

Results & Capabilities

Capability	Outcome
Input formats	5 video + 3 doc
OCR language	EN
Per-question checks	4, ≤ 3 retries
Quiz / study types	5
Grounding	100% + timestamp
LLM hosting	local, \$0 API
Cached OCR re-runs	skipped
Agent failures	isolated

Table 1. System capabilities. Eval metrics (acceptance rate, validator-vs-human, OCR/WER, T4 runtime) to be added.